

School of Computer Science and Engineering

Month, Year

MEETBRIEF AI: A PRIVACY-PRESERVING MULTILINGUAL MEETING ASSISTANT WITH SHARED ROOMS FOR LATE JOINERS

Submitted in partial fulfillment for the award of the degree of

Name of the degree

by

NAME OF THE CANDIDATE

(USN)

Guided by

GUIDE NAME



School of Computer Science and Engineering

Month, Year

MEETBRIEF AI: A PRIVACY-PRESERVING MULTILINGUAL MEETING ASSISTANT WITH SHARED ROOMS FOR LATE JOINERS

Submitted in partial fulfillment for the award of the degree of

Name of the degree

by

NAME OF THE CANDIDATE

(USN)

Guided by

GUIDE NAME



School of Computer Science and Engineering

RV University

RV Vidyaniketan, 8th Mile, Mysore Road, Bengaluru, Karnataka, India - 560059

CERTIFICATE

This is to certify that the project work titled "MEETBRIEF AI: A PRIVACY-PRESERVING MULTILINGUAL MEETING ASSISTANT WITH SHARED ROOMS FOR LATE JOINERS" is performed by NAME OF THE CANDIDATE (USN), a bonafide student of Name of the degree at the School of Computer Science and Engineering, RV University, Bengaluru in partial fulfillment for the award of degree Name of the degree in Computer Science & Engineering, during the Academic year **2026-2027**.

[Guide Name]	Dr. Rashmi S	Dr. G Shobha
Guide	[B.Tech(H)]	
[Assistant Professor]	Program Director	Dean
SoCSE	SoCSE	SoCSE
RV University	RV University	RV University
Date:	Date:	Date:

Signature of the Mentor	Signature of the Guide
Name of the Examiner	Signature of Examiner
1.	1.
2.	2.

DECLARATION

I hereby declare that the thesis entitled “MeetBrief AI: A Privacy-Preserving Multilingual Meeting Assistant with Shared Rooms for Late Joiners” submitted by me, for the award of the degree Name of the degree in RV University is a record of bonafide work carried out by me under the supervision of <Guide Name>.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Bangalore

Date: **Signature of the Candidate**

ABSTRACT

Online meetings on Google Meet, Zoom and Microsoft Teams are now the default way that classes and project teams work, yet a participant who joins late, loses the connection, or does not follow a technical term has no reliable way to recover what was said. Existing assistants are cloud services that send meeting content to external servers, are mostly English-first, and serve only the person who started them.

This project presents **MeetBrief AI**, a Chrome extension with a local Python backend that turns a live meeting into a structured, searchable and explainable record. Captions are read from the meeting page together with the tab audio (hybrid capture), and audio is transcribed with Whisper and separated by speaker with WavLM embeddings. A hybrid pipeline of transparent rules and open Hugging Face models running entirely on the user's machine extracts decisions, revisions, tasks, owners and deadlines; produces TextRank and BART-SAMSum summaries whose every sentence is aligned to transcript evidence; answers questions with hybrid retrieval and RoBERTa-SQuAD2 while abstaining when the answer is absent; explains statements with Qwen2.5-1.5B; and lets every viewer read the transcript in their own language, using Whisper speech translation and NLLB-200. *Shared rooms* keyed by the meeting link give every participant with the extension a personalised "what you missed" catch-up, with one elected capturer and automatic handover.

On annotated evaluation meetings the system reaches an action-item F1 of 1.0 and a held-out decision F1 of 0.80, answers 16 of 16 answerable questions while abstaining on 7 of 7 unanswerable ones, and raises overview ROUGE-1 from 0.36 to 0.55 with all generated sentences evidence-supported. Cue-phrase rules (accuracy 0.94) outperformed zero-shot NLI (0.59) for dialogue acts, and Qwen2.5-1.5B gave 8 of 9 correct explanations versus 0 of 9 for Flan-T5-large. Seventy-three automated tests, browser harnesses, a security scan and a live two-laptop Google Meet trial validated the system. The results show that a fully local, evidence-grounded meeting assistant is practical on a student laptop.

Keywords: meeting summarization, question answering, dialogue acts, speaker diarization, multilingual NLP, privacy-preserving AI.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to [Internal Guide name], [Designation, School of Computer Science and Engineering, RV University], for his/her constant guidance, continual encouragement, understanding; more than all, he/she taught me patience in my endeavor. My association with him/her is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of Natural Language Processing.

I would like to express my gratitude to Prof Dr Ram Kumar Kakani, Vice Chancellor RV University, and Prof Dr Shobha G, Dean SoCSE, RV University, for providing with an environment to work in and for his inspiration during the tenure of the course.

In jubilant mood I express ingeniously my whole-hearted thanks to Dr. Rashmi S, Program Director, [B.Tech(H)] SoCSE, RV University, all teaching staff and members working as limbs of our university for their not-self-centered enthusiasm coupled with timely encouragements showered on me with zeal, which prompted the acquirement of the requisite knowledge to finalize my course study successfully. I would like to thank my parents for their support.

I would like to extend my sincere thanks to my mentor, [Mentor Name], [Designation, School of Computer Science and Engineering, RV University], for their unwavering support, thoughtful guidance, and motivational presence throughout the course of this project. Their mentorship not only enriched my learning but also inspired me to approach challenges with confidence and clarity. Working with them has been a meaningful experience, and I am truly grateful for the valuable time and insights shared with me during this journey.

It is indeed a pleasure to thank my friends who persuaded and encouraged me to take up and complete this task, and who joined the live test meetings of MeetBrief. At last but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly toward the successful completion of this project.

Place: Bangalore **NAME OF THE CANDIDATE**

Date:

CONTENTS

ABSTRACT i
ACKNOWLEDGEMENT ii
CONTENTS iii
LIST OF FIGURES iv
LIST OF TABLES v
LIST OF ACRONYMS vi
Chapter 1: Introduction 1
1.1 INTRODUCTION 1
1.2 PROBLEM STATEMENT 2
1.3 OBJECTIVES 2
1.4 SCOPE OF THE PROJECT 3
1.5 ORGANISATION OF THE REPORT 4
Chapter 2: Background 5
2.1 LITERATURE SURVEY 5
2.2 TOOLS AND TECHNIQUES 8
Chapter 3: Methodology 10
3.1 PROPOSED METHODOLOGY 10
Chapter 4: Implementation 17
4.1 DATASETS 17
4.2 IMPLEMENTATION 17
4.3 EXPERIMENTAL SETUP 20
Chapter 5: Results 22
5.1 EVALUATION METRICS 22
5.2 RESULT SUMMARY TABLES 22
5.3 ANALYSIS OF RESULTS 26
Chapter 6: Inferences 29
6.1 KEY FINDINGS 29
6.2 FULFILLMENT OF OBJECTIVES 29
Chapter 7: Impression Report 31
7.1 PERSONAL LEARNING EXPERIENCE 31
7.2 SKILL ENHANCEMENT 31
7.3 INDUSTRY / ACADEMIC RELEVANCE 32
Chapter 8: Conclusion & Future Work 33
FUTURE WORK 33
References 35
Appendix 39
APPENDIX 1: SELECTED REST API ENDPOINTS 39
APPENDIX 2: PROJECT STRUCTURE AND HOW TO RUN 39
APPENDIX 3: EXAMPLE CATCH-UP OUTPUT (UNEDITED) 40

LIST OF FIGURES

Figure 3.1 System architecture of MeetBrief AI 10
Figure 3.2 Processing pipeline from captured input to user-facing features 11
Figure 3.3 Shared-room protocol: late joiner catch-up and capturer handover 15
Figure 4.1 MeetBrief side panel during a live Google Meet capture 18
Figure 4.2 On-page status label shown inside the meeting page 18
Figure 5.1 Overview summary quality: extractive TextRank vs abstractive BART-SAMSum 23
Figure 5.2 Dialogue-act recall of cue-phrase rules versus zero-shot NLI 24
Figure 5.3 Explanation quality of the three candidate models 24
Figure 5.4 Latency of the rule-based and model-based configurations 25

LIST OF TABLES

Table 2.1 Summary of the literature survey 7
Table 2.2 Tools, libraries and models used 8
Table 4.1 Datasets used for development and evaluation 17
Table 4.2 Excerpt of MeetBrief output for the sprint-planning meeting 19
Table 4.3 Experimental setup 21
Table 5.1 Information extraction and Q&A (rules only vs with Hugging Face models) 23
Table 5.2 Summary quality (ROUGE F1) and faithfulness 23
Table 5.3 Summarizer selection benchmark (development set) 23
Table 5.4 Dialogue-act classification on 32 labelled sentences 23
Table 5.5 Explanation model benchmark (9 items, human-rated) 24
Table 5.6 Speech and translation 25
Table 5.7 Latency in milliseconds (held-out meeting, CPU) 25
Table 5.8 Issues found in the live trial and their fixes 25
Table 5.9 Comparison with existing approaches 27
Table 6.1 Fulfillment of the objectives 30
Table A.1 Selected backend endpoints 39

LIST OF ACRONYMS

AMI	Augmented Multi-party Interaction corpus	**NER**	Named Entity Recognition
API	Application Programming Interface	**NLI**	Natural Language Inference
ASR	Automatic Speech Recognition	**NLLB**	No Language Left Behind (MT)
BART	Bidirectional Auto-Regressive Transformer	**NLP**	Natural Language Processing
BERT	Bidirectional Encoder Repr. from Transformers	**OAuth**	Open Authorization
BOLA	Broken Object-Level Authorization	**QA**	Question Answering
chrF	Character n-gram F-score	**RAG**	Retrieval-Augmented Generation
CORS	Cross-Origin Resource Sharing	**REST**	Representational State Transfer
CSP	Content Security Policy	**ROUGE**	Recall-Oriented Understudy for Gisting Eval.
DAST	Dynamic App. Security Testing	**SAMSum**	Samsung dialogue summary corpus
DER	Diarization Error Rate	**SQuAD**	Stanford Question Answering Dataset
DOM	Document Object Model	**TF-IDF**	Term Freq.–Inverse Document Freq.
F1	Harmonic mean of precision, recall	**TTS**	Text-to-Speech
HF	Hugging Face	**UI**	User Interface
JSON	JavaScript Object Notation	**URL**	Uniform Resource Locator
LLM	Large Language Model	**VAD**	Voice Activity Detection
MMR	Maximal Marginal Relevance	**VTT**	Web Video Text Tracks
MPS	Metal Performance Shaders (GPU)	**WER**	Word Error Rate
MV3	Chrome Manifest Version 3	**WavLM**	Wave Language Model (speech)

Chapter 1: Introduction

1.1 INTRODUCTION

Meetings are where most decisions in teams are made and most tasks are assigned. Since 2020, a large share of these meetings moved to video platforms such as Google Meet, Zoom and Microsoft Teams. The amount of spoken information exchanged in a one-hour call is large, it is rarely written down completely, and the people who most need a recap — participants who joined late, dropped out because of a weak connection, or attended a meeting in a language they are less comfortable with — are exactly the ones who cannot ask the others to repeat everything.

Natural Language Processing (NLP) offers the building blocks to solve this: speech recognition, summarization, information extraction, question answering, machine translation and text simplification. Large commercial assistants combine these as cloud services. However, for students and many organisations three limitations remain: (i) meeting content is sent to third-party servers, which raises privacy and consent concerns; (ii) the tools are mostly English-first and do not handle Indian code-switched speech such as Hinglish or Telugu–English well; and (iii) the assistant usually serves only the person who started it, so a participant who joins late gets nothing for the minutes they missed.

This project, **MeetBrief AI**, is a Chrome extension with a local backend that addresses these limitations. It captures what is said in a meeting, builds a structured *meeting memory* (decisions, action items, owners, deadlines, open questions and topics), and lets the user catch up, ask questions, request an easier explanation of any statement, translate the recap into Hindi or Telugu, and export the result. All neural models run locally from the Hugging Face Hub, and every generated sentence is linked to the transcript lines that support it.

1.1.1 MOTIVATION

The idea came from a common experience in college project meetings: someone joins ten minutes late and asks "what did I miss?", interrupting the meeting, or silently stays confused. A recap that is generated automatically, personalised to the moment each person joined, and grounded in the actual words spoken would remove this friction. Because the project is part of an NLP course, a second motivation was to study how far *small, open* models can go when they are combined carefully with classical, explainable techniques instead of relying on a paid API.

1.1.2 THE LATE-JOINER PROBLEM

No software can recover speech that was never captured. A late joiner can only receive a recap if (a) some other participant's device was capturing the meeting from the start, or (b) the meeting platform itself recorded a transcript. MeetBrief solves case (a) with *shared rooms*: the first participant with the extension registers the meeting link, captures it, and every later participant with the extension automatically joins the same room and receives everything said before they arrived. Case (b) is handled by importing the official transcript of Google Meet, Zoom or Microsoft Teams, or a recording file. Meetings held in desktop applications are covered by a desktop recorder that sends system audio to the same pipeline.

1.2 PROBLEM STATEMENT

To design and implement a privacy-preserving, multilingual meeting assistant that (1) captures online meetings from captions or audio without sending data to external AI services, (2) extracts decisions, action items, owners, deadlines and unresolved questions with traceable evidence, (3) produces faithful summaries and answers questions about the meeting while refusing to guess when the information is absent, (4) explains and translates meeting content for participants with different backgrounds and languages, and (5) gives participants who join late a personalised catch-up of what they missed, using only open models that run on a student laptop.

1.3 OBJECTIVES

- **O1 – Capture:** read live captions from Google Meet (and experimentally Zoom and Teams web), transcribe tab or desktop audio with speaker separation, and import transcripts or recordings.

- **O2 – Understanding:** build a structured meeting memory: dialogue acts, decisions with revision history, tasks with owners and deadlines, open questions, topics, entities and potential contradictions.
- **O3 – Faithful summaries:** generate catch-up, quick, detailed, chronological, topic-wise and personalised summaries in which every sentence is aligned to supporting transcript segments.
- **O4 – Grounded Q&A:** answer natural-language questions about the meeting with quoted evidence and abstain when the transcript does not contain the answer.
- **O5 – Explain and translate:** explain any statement at four difficulty levels in five styles, and translate outputs into each viewer's language, with English, Telugu, Kannada, Hindi and Tamil as the meeting languages.
- **O6 – Late joiners:** implement shared rooms with per-member "what you missed" catch-up, a single elected capturer and automatic handover, plus Early Join and official-transcript import.
- **O7 – Privacy and security:** run all models locally, support private (in-memory) mode, retention and deletion, and validate the API with automated security testing.
- **O8 – Evaluation:** measure every component (ROUGE, precision/recall/F1, QA accuracy and abstention, WER, speaker accuracy, chrF, latency) and compare rule-based and model-based variants.

1.4 SCOPE OF THE PROJECT

In scope. A Chrome Manifest V3 extension (side panel, service worker, content scripts, offscreen audio recorder), a FastAPI backend with SQLite storage, local Hugging Face models for summarization, translation, explanation, question answering, embeddings, NER, zero-shot NLI and speaker embeddings, Whisper ASR, shared rooms for multi-participant use on a common server, connectors for official transcripts, a desktop recorder, an evaluation framework and a test suite. English, Telugu, Kannada, Hindi, Tamil and code-switched input are supported, with Kannada the weakest (Section 3.1.8).

Out of scope. Recovering speech that nobody captured; a server-side meeting bot that joins calls without any participant's browser; native desktop or mobile apps; and large-scale evaluation on real recorded corpora such as AMI or ICSI (the framework supports QMSum, but the reported numbers come from small annotated synthetic meetings).

1.5 ORGANISATION OF THE REPORT

Chapter 2 reviews prior work on meeting understanding year by year and lists the tools used. Chapter 3 describes the proposed methodology and architecture. Chapter 4 covers datasets, implementation and the experimental setup. Chapter 5 reports quantitative results and compares the approach with existing methods. Chapter 6 discusses key findings and how the objectives were met, Chapter 7 is the impression report, and Chapter 8 concludes with future work. References and appendices follow.

Chapter 2: Background

2.1 LITERATURE SURVEY

Research on automatic meeting understanding spans speech processing, dialogue analysis, summarization and question answering. The survey below is organised chronologically to show how methods moved from hand-crafted features to pre-trained Transformers, and which gaps remain for a local, multilingual assistant.

2.1.1 EARLY WORK (2000–2007): DIALOGUE ACTS, CORPORA AND EXTRACTIVE SUMMARIES

Stolcke et al. (2000) modelled dialogue acts — statements, questions, agreements — with hidden Markov models over word and prosodic features on Switchboard, showing that the function of an utterance can be recognised automatically. The ICSI Meeting Corpus (Janin et al., 2003) and the AMI Meeting Corpus (Carletta et al., 2005) provided the first large multi-party meeting recordings with transcripts and annotations, and they remain the main benchmarks. Mihalcea and Tarau (2004) introduced TextRank, a graph-based ranking of sentences derived from PageRank (Page et al., 1999), which became a strong unsupervised extractive baseline. Hearst (1997) had earlier proposed TextTiling for segmenting text into topical units using lexical cohesion. Murray, Renals and Carletta (2005) compared extractive summarization techniques on meeting recordings and found that meeting speech, with disfluencies and overlapping turns, is much harder than news text.

2.1.2 DECISIONS AND ACTION ITEMS (2007–2010)

Purver et al. (2007) detected and summarised action items in multi-party dialogue by first classifying the utterances that describe a task, an owner, a timeframe or an agreement, and Hsueh and Moore (2007) detected decision-related dialogue acts in AMI meetings. The CALO Meeting Assistant (Tur et al., 2010) integrated speech recognition, topic segmentation, action-item and decision detection into one system. These works established that meeting outcomes are better represented as structured records than as plain summaries — an idea MeetBrief adopts for its meeting memory — but they relied on corpus-specific supervised classifiers and server-side processing.

2.1.3 TRANSFORMERS AND PRE-TRAINING (2017–2019)

The Transformer (Vaswani et al., 2017) and pre-trained encoders such as BERT (Devlin et al., 2019) and RoBERTa (Liu et al., 2019) changed nearly every NLP task. For question answering, SQuAD 2.0 (Rajpurkar, Jia and Liang, 2018) added unanswerable questions, so that models learn to output *no answer* — the property MeetBrief uses for abstention. Sentence-BERT (Reimers and Gurevych, 2019) made dense sentence embeddings efficient for semantic search, and its multilingual distillation (Reimers and Gurevych, 2020) extended this to 50+ languages. Yin, Hay and Roth (2019) showed that NLI models can perform zero-shot text classification by treating labels as hypotheses. In speech, x-vector embeddings (Snyder et al., 2018) became the standard representation for speaker recognition and diarization. The SAMSum corpus (Gliwa et al., 2019) provided human-written summaries of chat dialogues.

2.1.4 ABSTRACTIVE AND RETRIEVAL-AUGMENTED METHODS (2020–2021)

BART (Lewis et al., 2020a), a denoising sequence-to-sequence model, achieved strong abstractive summarization results; fine-tuned on SAMSum it summarises dialogues rather than news. HMNet (Zhu et al., 2020) designed a hierarchical network with cross-domain pre-training specifically for meeting summarization. QMSum (Zhong et al., 2021) introduced query-based meeting summarization over AMI, ICSI and parliamentary meetings. At the same time, Maynez et al. (2020) showed that abstractive summarizers frequently hallucinate content not supported by the source, and Retrieval-Augmented Generation (Lewis et al., 2020b) proposed grounding generation in retrieved passages. Together these results motivate MeetBrief's design: use abstractive models for fluency, but retrieve and align evidence for every generated claim.

2.1.5 LARGE MULTILINGUAL AND SPEECH FOUNDATION MODELS (2022–2024)

WavLM (Chen et al., 2022) learned speech representations that work well for speaker verification and diarization, and Park et al. (2022) surveyed deep-learning diarization. NLLB-200 (NLLB Team, 2022) covered 200 languages including Hindi and Telugu with a single translation model. Whisper (Radford et al., 2023) trained on 680,000 hours of weakly supervised audio and became a robust multilingual ASR model. Small instruction-tuned language models such as Qwen2.5 (Qwen Team, 2024) can now run on a laptop and follow concise prompts for explanation and simplification. These advances make a fully local assistant feasible, but no surveyed system combines them with structured meeting memory, evidence alignment, abstention, Indian-language support and multi-participant catch-up.

Table 2.1: Summary of the literature survey

Year	Work	Method	Relevance / gap for this project
2000	Stolcke et al.	HMM dialogue-act tagging	Utterance function is learnable; needs labelled data
2003–05	ICSI / AMI corpora	Annotated meeting recordings	Standard benchmarks; English only
2004	Mihalcea & Tarau	TextRank (graph ranking)	Unsupervised extractive baseline (used)
2005	Murray et al.	Extractive meeting summaries	Speech disfluencies make meetings hard
2007	Purver et al.; Hsueh & Moore	Action-item / decision detection	Structured outcomes; corpus-specific
2010	Tur et al. (CALO)	Integrated meeting assistant	Server-side, no late-joiner support
2018	Rajpurkar et al.	SQuAD 2.0 unanswerable questions	Enables abstention (used)
2019	Devlin et al.; Reimers & Gurevych	BERT; Sentence-BERT	Encoders and semantic search (used)
2019	Yin et al.	Zero-shot NLI classification	Tested; weaker than rules here
2020	Lewis et al.; Maynez et al.	BART; hallucination study	Abstractive + need for grounding
2021	Zhong et al.	QMSum query-based summaries	Supported by the evaluation tool
2022	Chen et al.; NLLB Team	WavLM; NLLB-200	Diarization and Hindi/Telugu MT (used)
2023	Radford et al.	Whisper ASR	Local multilingual ASR (used)
2024	Qwen Team	Qwen2.5 small LLMs	Local explanations (used)

2.1.6 RESEARCH GAP

Prior research systems treat the meeting as a single recording processed after the fact, usually on a server, and evaluate summarization quality in English. Commercial assistants add live capture but are cloud-based and single-user. The gap addressed by this project is an assistant that is (1) live and multi-participant, giving each late joiner their own catch-up; (2) local and privacy-preserving; (3) evidence-grounded with explicit abstention; and (4) usable in Indian multilingual settings, while remaining explainable enough for an academic evaluation of each NLP component.

2.2 TOOLS AND TECHNIQUES

Table 2.2 lists the main tools used. All of them are open source and free; no paid API is used anywhere in the system.

Table 2.2: Tools, libraries and models used

Category	Tool / model	Purpose in MeetBrief
Language / runtime	Python 3.13, JavaScript (ES2022)	Backend and Chrome extension
Web framework	FastAPI, Uvicorn, Pydantic	REST API, validation, auth middleware
Storage	SQLite (disk + in-memory)	Meetings, segments, rooms, notes; private mode
Browser platform	Chrome Manifest V3	Side panel, service worker, content scripts, offscreen audio, identity, alarms
Model hub	Hugging Face Transformers, Sentence-Transformers	Local model loading and inference (CPU / Apple MPS)
Summarization	philschmid/bart-large-cnn-samsum	Abstractive dialogue summaries
Question answering	deepset/roberta-base-squad2	Extractive QA with no-answer score
Embeddings	paraphrase-multilingual-MiniLM-L12-v2	Dense retrieval, topic labels, evidence alignment
Explanation	Qwen/Qwen2.5-1.5B-Instruct	Explain Again (simplify, example, analogy, …)
Translation	facebook/nllb-200-distilled-600M	English ↔ Hindi / Telugu and 200 languages
NER	dslim/bert-base-NER	Entities; verifying task owners
Zero-shot NLI	MoritzLaurer/mDeBERTa-v3-base-mnli-xnli	Optional dialogue-act second opinion
Speech	faster-whisper (Whisper small), microsoft/wavlm-base-plus-sv	ASR and speaker embeddings
Classical NLP	TF-IDF, TextRank, TextTiling, MMR, langdetect	Fallbacks and explainable components
Export	ReportLab (PDF), Markdown	Meeting minutes export
Testing / security	pytest, browser harnesses, StackHawk HawkScan	Unit, integration and DAST testing

Techniques. The system combines classical techniques — regular-expression and cue-phrase rules, TF-IDF, graph ranking, lexical cohesion and online clustering — with neural ones: encoder–decoder Transformers for summarization and translation, an encoder for extractive QA, a decoder-only LLM for explanations, bi-encoders for embeddings and self-supervised speech models. The guiding principle is *hybrid*: precise structured facts come from transparent rules with evidence, fluent text comes from models, and every model output is verified against the transcript.

Chapter 3: Methodology

3.1 PROPOSED METHODOLOGY

MeetBrief is organised in four layers (Figure 3.1): a **capture layer** in the meeting tab, a **coordination layer** in the extension's service worker, a **local backend** that runs the NLP pipeline and stores the meeting memory, and a **presentation layer** in Chrome's side panel. The design follows four principles.

- **Local first.** Every model runs on the user's machine; raw audio is deleted as soon as it is transcribed.
- **Evidence over fluency.** Every summary sentence, answer and extracted item carries the ids of the transcript segments that support it, and unsupported generated sentences are flagged.
- **Graceful degradation.** Each neural component has a classical fallback, so the system still works if a model is missing or the laptop is slow.
- **Abstain rather than guess.** When the transcript does not contain an answer, the system says so.

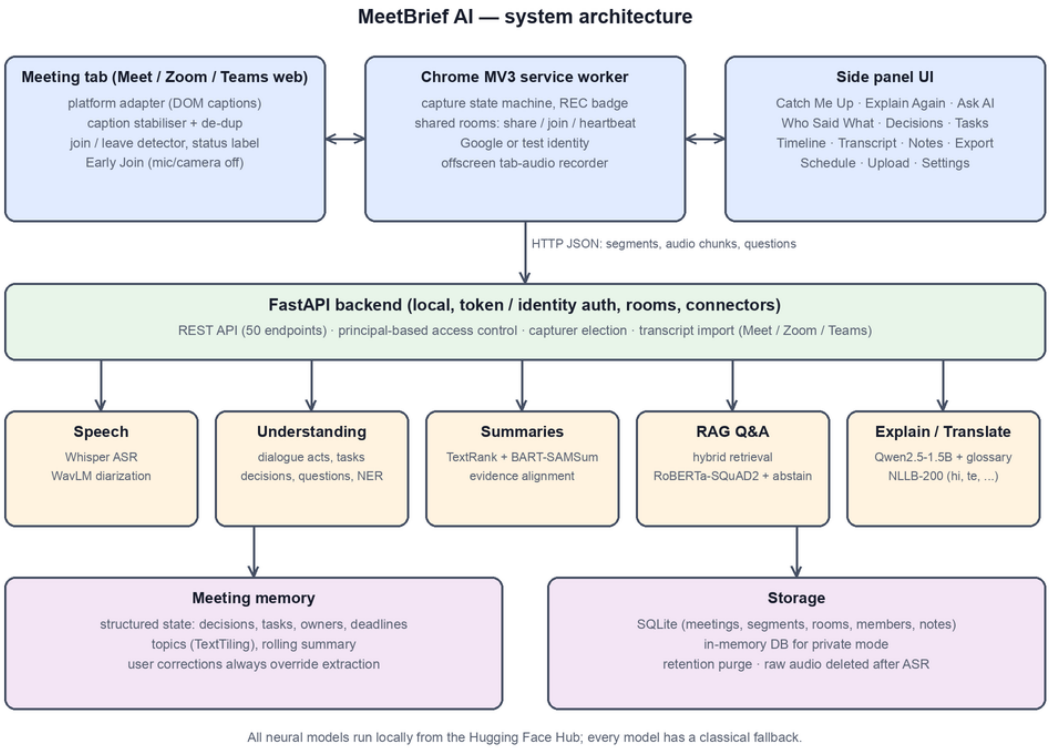


Figure 3.1: System architecture of MeetBrief AI

3.1.1 CAPTURE LAYER

A platform adapter for each meeting app reads the accessible caption region of the page. Captions are updated progressively ("We need" → "We need to" → "We need to improve…"), so the capture module emits a block only after its text has been stable for 1.8 s, sends only the new words beyond what was already emitted, and filters out text that belongs to the platform's own buttons and icons. The adapter also detects when the user joins and leaves the call (using the call controls and participant tiles, with a 6-second debounce because Meet hides its toolbar when the mouse is idle), and shows a small on-page status label. When captions are unavailable, the user can capture the tab's audio: an offscreen document records 15-second WebM/Opus chunks that the backend transcribes. For meetings in desktop apps, a command-line recorder streams a loopback or microphone device to the same endpoint. Official transcripts can be imported from Google Meet, Zoom cloud recordings and Microsoft Teams.

3.1.2 TEXT NORMALISATION AND LANGUAGE IDENTIFICATION

All sources are converted into a common *segment* schema (text, start and end time, speaker, language, source and flags). The normaliser parses VTT, SRT, JSON, DOCX and plain-text transcripts, attaches timestamps, removes fillers and merges fragments. Language is identified per segment by combining Unicode script ranges (Telugu, Devanagari, …), a lexicon of romanised Hindi and Telugu words for code-switched text, and a character n-gram naive-Bayes detector.

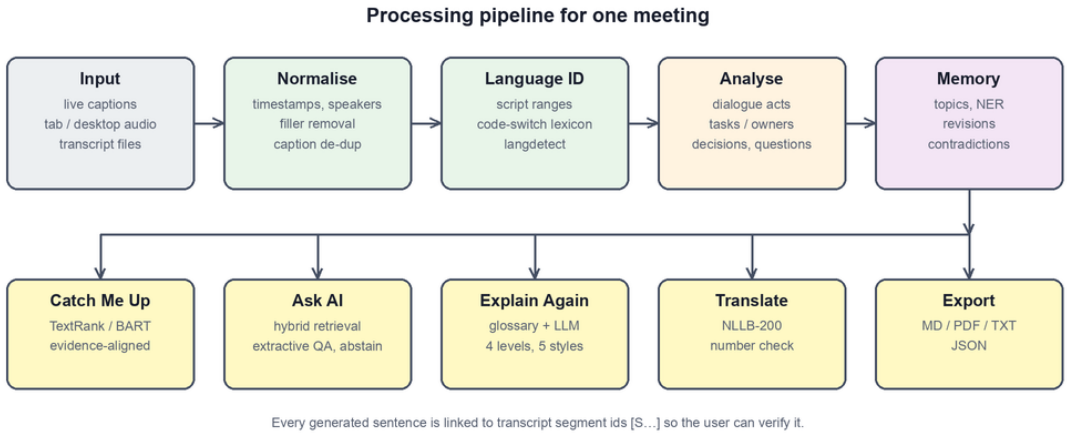


Figure 3.2: Processing pipeline from captured input to user-facing features

3.1.3 DIALOGUE ACTS AND INFORMATION EXTRACTION

Each sentence is classified into *decision*, *proposal*, *task*, *question* or *other* using cue-phrase rules (for example "let's go with", "we decided", "please … by Friday", "I will …"). An optional second opinion from a zero-shot NLI model (hypotheses such as "The speaker makes a decision") can raise the confidence of a rule-based item but never adds items on its own, because evaluation showed that it confuses deciding with suggesting. Tasks are extracted with their owner (an addressee such as "Karthik, please…", or a self-commitment "I will…") and deadline; owners are verified with BERT-based NER. A simple coreference step merges vague confirmations such as "Sure, I can do that by Friday" into the request they answer. A later confirmed decision that contains a revision cue ("actually", "instead", "postpone") on the same subject marks the earlier decision as *superseded*, so the memory keeps the history instead of overwriting it. Questions are paired with later replies, and unanswered ones are reported.

3.1.4 TOPIC SEGMENTATION AND MEETING MEMORY

Topics are found with TextTiling: a depth score is computed over the cosine similarity of TF-IDF windows, combined with transition cues ("moving on to…"); each topic receives a KeyBERT-style label chosen by embedding similarity and Maximal Marginal Relevance. The meeting memory stores topics, decisions with revisions, tasks, questions, entities and potential contradictions (high-similarity sentence pairs that differ in polarity or numbers). It is recomputed from the stored transcript and cached by a fingerprint, while user corrections (task status, owner, deadline, speaker names) are stored separately and always override extracted values.

3.1.5 SUMMARIZATION WITH EVIDENCE ALIGNMENT

Two summarizers are combined. TextRank ranks sentences by PageRank over a TF-IDF cosine-similarity graph with boosts for problem and reason statements; it provides *key moments* that are quoted verbatim. BART fine-tuned on SAMSum generates an abstractive *overview*; long meetings are processed hierarchically in chunks under the 1,024-token limit. Each generated sentence is then aligned to its most similar transcript segment with dense embeddings; sentences with low similarity, or with entities that never occur in the transcript, are flagged with a warning marker. The summary also lists options discussed, confirmed and superseded decisions, action items and the current topic. Summary modes include catch-up, quick, detailed, chronological, by-topic, personalised (for one speaker), "last N minutes" and a final summary built automatically when the meeting ends.

3.1.6 RETRIEVAL-AUGMENTED QUESTION ANSWERING WITH ABSTENTION

A question is first rewritten using the chat history for follow-ups ("and who will do it?"). Retrieval combines word-level TF-IDF, character 3–5-gram TF-IDF (robust to spelling and romanisation) and dense multilingual embeddings, with optional speaker and time filters. A relevance gate requires the retrieved passages to cover at least half of the question's content words; "why" questions additionally need an explicit reason cue. Questions about owners, deadlines or decisions are answered from the structured memory; others go to RoBERTa fine-tuned on SQuAD 2.0, whose span is accepted only if it beats the *no-answer* score. If an answer comes from a decision that was later revised, the reader is re-run on the revision so that the current state leads (for example "Tuesday — earlier plan: Friday"). Otherwise the system replies that the information is not in the captured transcript.

3.1.7 EXPLAIN AGAIN AND TRANSLATION

Explain Again shows what the meeting itself said about a selected term (retrieved segments) and then an explanation in one of five styles (simplify, example, step-by-step, detailed, analogy) at four levels. A curated glossary is used first; otherwise Qwen2.5-1.5B-Instruct generates a short explanation that is clearly labelled as general knowledge to verify. Outputs can be translated with NLLB-200 while preserving Markdown, segment citations and numbers (a post-check confirms that every number in the source appears in the translation).

3.1.8 SPEECH RECOGNITION AND SPEAKER DIARIZATION

Audio chunks are transcribed by Whisper (faster-whisper, small model) with voice-activity filtering and word timestamps. Audio is split on pauses and the language of each utterance is detected, limited to the configured meeting languages (by default English, Telugu, Kannada, Hindi and Tamil), so a speaker may switch language inside a chunk. For non-English speech Whisper also translates the speech directly into English; the rule and model components read this English, while the original words are kept for display. This was chosen after testing: Whisper small writes Telugu and Kannada text unreliably but translates the speech far better, so those two languages are shown in English only, tagged with the language they were translated from. Each viewer picks a reading language, and other targets are produced with NLLB-200. Short phrases Whisper invents on silence are filtered. On a text-to-speech test track, English, Telugu, Hindi and Tamil were usable and Kannada was the weakest; this is an informal check, not a measured accuracy.

Captions and tab audio are captured together. Captions give real names and include the capturing user's own voice, which tab audio never contains. An audio sentence with at least 60% content-word overlap with one recent caption line is dropped as a duplicate; the rest is kept, so nothing is lost when captions stop. Two agreeing matches also map a voice label to the caption's name. Chrome grants tab capture only after the user invokes the extension on the tab, through its toolbar icon, shortcut or context menu.

For each recognised segment a WavLM-Base-Plus x-vector is computed, and an *online* agglomerative clustering assigns it to the most similar existing speaker centroid if the cosine similarity exceeds a threshold (0.86), otherwise a new speaker is created; centroids persist across chunks so labels stay consistent through the meeting. Generic labels (Speaker 1, Speaker 2…) are never replaced by guessed real names; the user maps them.

3.1.9 SHARED ROOMS FOR LATE JOINERS

The key used to identify a meeting is derived from its link (for example *google_meet:abc-defg-hij*, handling both *meet.google.com/abc-defg-hij* and *meet.google.com/meet/abc-defg-hij*). When a signed-in user joins a call, the extension asks the server whether a room exists for that key. If not, it offers to *share* the meeting; the sharer becomes the room's capturer. Every other member who joins the same call is admitted automatically (optionally only for the sharer's email domain or after approval), stands by, and receives a catch-up computed up to *their own* join time (Figure 3.3). Members send a heartbeat every 10 s; the server keeps one capturer, prefers a member whose captions are visible if the capturer reports captions off, and hands over capturing when the capturer leaves or misses heartbeats for 25 s. Rooms idle for 15 minutes are closed so that a reused meeting link starts a new room, and when the last member leaves, the room ends with a final summary.

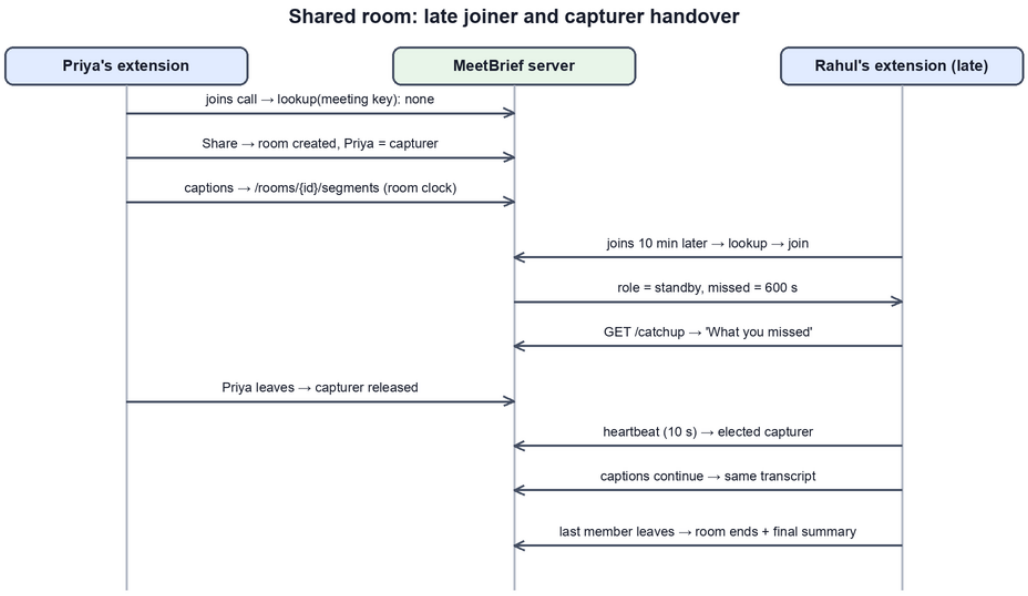


Figure 3.3: Shared-room protocol: late joiner catch-up and capturer handover

3.1.10 PRIVACY AND SECURITY DESIGN

The backend binds to localhost by default and requires an API token compared in constant time. Room members are identified by a Google token verified by the server (audience = the project's OAuth client) or, in test mode, by a typed identity. Access control is principal-based and fail-closed: members may call only an allow-list of read and ask routes, may read only meetings of rooms they belong to (other ids return 404 to prevent probing), may not delete or modify meetings, and only the current capturer may push transcript segments. Other members' email addresses are visible only to the sharer. Private mode keeps a meeting in an in-memory database that is never written to disk, retention is configurable, and the web app uses a nonce-based Content Security Policy.

Chapter 4: Implementation

4.1 DATASETS

Because public meeting corpora are English-only and large, the project created small, fully annotated synthetic meetings that exercise every feature, including Indian code-switching. Two annotation files separate a *development* set (used while writing the rules, so its scores are optimistic) from a *held-out* set annotated before the first evaluation run. Table 4.1 summarises the data.

Table 4.1: Datasets used for development and evaluation

Dataset	Content	Annotations / use
sprint_planning.txt	Product sprint planning (5 speakers, ~6 min)	Decisions incl. one revision, tasks, owners, deadlines, Q&A
demo_indexing.txt	Database performance discussion	Decisions, proposals, tasks, Q&A
mixed_hinglish.txt	Hindi–English code-switched meeting	Language ID, translation, extraction
telugu_english.txt	Telugu–English code-switched meeting	Language ID, translation, extraction
unlabeled_speakers.vtt, sample.srt, sample.json	Same content in caption formats	Parser and normaliser tests
heldout_design_review.txt	Notification-system design review	Held-out decisions, tasks, questions, Q&A
dialogue_acts.json	32 hand-labelled sentences	Rules vs zero-shot NLI comparison
three_speakers.wav	29.3 s synthetic meeting, 3 TTS voices	WER and speaker-accuracy evaluation
QMSum (optional, not bundled)	Query-based summaries of AMI/ICSI meetings	Supported via --qmsum for larger studies

4.2 IMPLEMENTATION

The implementation consists of about 7,600 lines of Python in the backend and tools and about 2,700 lines of JavaScript, HTML and CSS in the extension. The backend exposes 50 REST endpoints.

4.2.1 CHROME EXTENSION

- **Content scripts** (`meeting_detector.js`, `caption_capture.js`, platform adapters for Google Meet, Zoom web and Teams web): call detection, caption stabilisation, on-page status label (Figure 4.2), Early Join (opens a scheduled meeting one minute early and joins with microphone and camera off).

- **Service worker** (`background.js`): per-tab capture state machine serialised through a single queue, REC/ROOM badge, notifications with Share/Not-now buttons, room join/share/heartbeat, routing of transcript segments to the meeting or room, alarms for Early Join, and re-injection into meeting tabs that were open before an update.
- **Side panel** (Figure 4.1): tabs for Catch Me Up, Explain Again, Ask AI, Who Said What, Decisions, Action Items, Timeline, Transcript, Notes, Schedule, Upload and Settings, built with strict-CSP-safe JavaScript.
- **Offscreen document**: records only the meeting tab's audio after the user explicitly starts it (no microphone permission is requested).

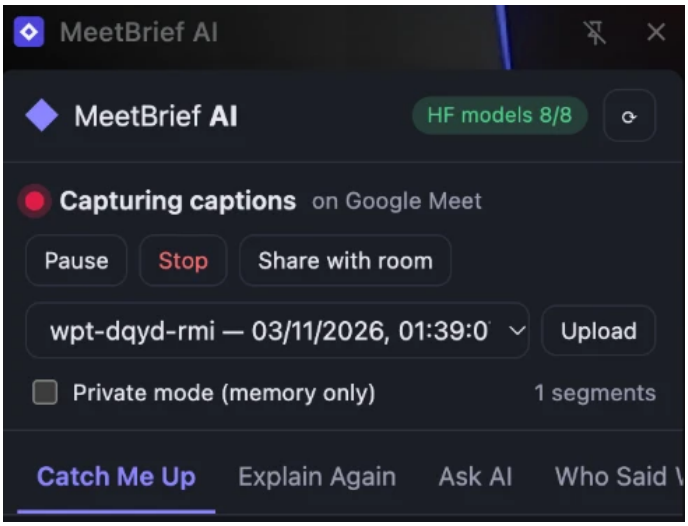


Figure 4.1: MeetBrief side panel during a live Google Meet capture

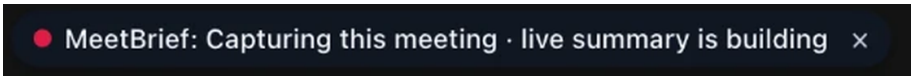


Figure 4.2: On-page status label shown inside the meeting page

4.2.2 BACKEND

- `api/` – routes, rooms and service layer; `auth.py` – principals (owner token, Google identity, test identity) and the fail-closed access middleware.
- `capture/` – transcript normaliser and caption de-duplication; `speech/` – Whisper ASR, WavLM diarization, alignment.
- `nlp/` – language identification, dialogue acts, decisions, actions, questions, topics, summarization, evidence alignment, simplification, glossary, translation, speakers, contradictions.

- `rag/` – hybrid embeddings, retrieval and answer generation; `memory/` – meeting state, rolling summary, decision history.
- `hf/` – local model registry with lazy loading, per-task fallback, device selection (CPU, CUDA or Apple MPS) and a semaphore that limits concurrent heavy jobs; `database/` – SQLite with a re-entrant lock serialising access; `connectors/` – Google Meet REST API, Zoom cloud recording and Microsoft Graph transcript import.
- `evaluation/` – metrics implemented from scratch (ROUGE, P/R/F1, WER, chrF, speaker accuracy) and the benchmark scripts; `export/` – Markdown, PDF (with bundled Noto fonts for Telugu and Hindi), TXT and JSON.

4.2.3 KEY ALGORITHMS

Caption stabilisation. For each caption block the module remembers the text already emitted; when the block has not changed for 1.8 s, disappears, or grows beyond 600 characters, only the words after the longest common word prefix are sent. The backend merges growing duplicate updates into the last segment, so de-duplication happens twice.

Hybrid retrieval score. For a query q and segment s, the score is a weighted combination of word TF-IDF cosine, character n-gram TF-IDF cosine and dense embedding cosine, followed by the relevance gate on content-word coverage. Character n-grams make retrieval robust to romanised Hindi and Telugu spellings.

Capturer election. On each heartbeat the server records the member's last-seen time and caption visibility. If the current capturer is alive (in the call and seen within 25 s) it stays; otherwise the first member in the call whose heartbeat arrives becomes the capturer. A member with visible captions takes over a capturer that explicitly reports captions off. Transcript segments are accepted only from the current capturer and are placed on the room's shared clock, so the transcript has neither gaps nor duplicates.

4.2.4 SAMPLE OUTPUT

Running the system on the sprint-planning meeting produced the following (real, unedited) results:

Table 4.2: Excerpt of MeetBrief output for the sprint-planning meeting

Feature	Output
Overview (BART)	"The payment page needs to be lazy loaded only when the user selects a payment method and the address validation should be moved to the backend. [S3] … The mobile app release for 2.4 will be postponed to next Tuesday. [S13]"
Decisions	Confirmed: go with lazy loading the payment scripts first (Meera) [S5]; postpone the 2.4 release to next Tuesday (Meera) [S13], superseding "release version 2.4 on Friday" [S10]
Action items	Implement the lazy loading — Karthik, Wednesday; Investigate the Android crash reports — David, today; Write the rollback plan — Sana, next Monday; Update the release notes — owner unknown (flagged)
Ask AI	Q: When is the 2.4 release now? A: "Tuesday" — Meera (04:12) [S13] (earlier plan: Friday, revised) Q: What is the marketing budget? A: "I couldn't find anything about this in the captured transcript."
Explain Again	"Lazy loading is a technique … where resources or data are loaded only when they are needed, rather than all at once …" — labelled as general knowledge (Qwen2.5-1.5B)
Translation (te)	మొబైల్ అప్లికేషన్ 2.4 విడుదల వచ్చే మంగళవారం వరకు వాయిదా పడింది.

4.2.5 TESTING AND QUALITY ASSURANCE

The project has 73 automated pytest tests: unit and API tests (concurrency, error handling, uploads, recordings, privacy, retention), 14 room tests (late joiner, access rules, approval, broken object-level authorization, capturer handover, idle-room expiry), connector tests with mocked provider APIs, desktop-recorder tests, and model tests that run every Hugging Face model. Browser harnesses run the real extension code against a simulated Meet page (17 checks: join, auto-captions, caption emission, toolbar flicker, leave, Early Join) and against two simulated members on a test server (12 checks: share, late-join catch-up, duplicate refusal, handover, room end). A dynamic security scan (StackHawk HawkScan) was run on a throwaway instance; it found a race condition that produced HTTP 500 errors (fixed by serialising database access), a permissive CSP (replaced with a nonce-based policy) and member email exposure in room views (restricted to the sharer); the final scan reported zero open findings.

4.3 EXPERIMENTAL SETUP

Table 4.3: Experimental setup

Item	Configuration
Hardware	Apple MacBook Air, Apple M2, 8-core CPU, MPS GPU (generation); other models on CPU
Software	macOS, Python 3.13, PyTorch, Transformers, Sentence-Transformers, faster-whisper, FastAPI, Chrome
Models	BART-large-CNN-SAMSum, RoBERTa-base-SQuAD2, MiniLM-L12 multilingual, Qwen2.5-1.5B-Instruct, NLLB-200-distilled-600M, BERT-base-NER, mDeBERTa-v3-base-XNLI, WavLM-base-plus-sv, Whisper small (ASR evaluation below used base)
Decoding	Greedy decoding for generation; NLLB beam size 4; summary chunks ≤ 1,024 tokens
Thresholds	Speaker cosine 0.86; QA minimum span score 0.15 and positive margin over no-answer; caption stability 1.8 s
Configurations	(a) rules only (MEETBRIEF_MODELS=off) and (b) with Hugging Face models; optional hybrid NLI
Live trial	Two laptops on the same Wi-Fi, Google Meet call, backend on one laptop (MEETBRIEF_HOST=0.0.0.0)

Every text component was measured twice, rules only versus with the Hugging Face models, so that the evaluation doubles as a model-selection benchmark. Summaries were compared with author-written references using ROUGE-1/2/L F1. Extraction was evaluated with precision, recall and F1 against hand annotations, owners and deadlines with accuracy on matched tasks, and Q&A with answer accuracy on answerable questions and correct abstention on unanswerable ones. Speech used word error rate and speaker accuracy after optimal label mapping, and translation used round-trip chrF with a number-preservation check. Latency was measured end to end on the backend.

Chapter 5: Results

This chapter reports the measured results. All numbers were produced by the project's evaluation script on the machine described in Section 4.3. The datasets are small and synthetic, so the results are a sanity check and a rules-versus-models comparison, not a general accuracy claim.

5.1 EVALUATION METRICS

- **ROUGE-1/2/L F1** (Lin, 2004): unigram, bigram and longest-common-subsequence overlap between a generated summary and a reference summary.
- **Precision, recall, F1** for decisions, proposals, action items and unresolved questions, using fuzzy matching of extracted items to annotated ones; **owner and deadline accuracy** on matched tasks.
- **Q&A accuracy** on answerable questions and **abstention rate** on unanswerable questions.
- **Evidence support**: share of abstractive sentences aligned to a supporting transcript segment.
- **WER** for speech recognition and **speaker accuracy** after optimal mapping of predicted to true speakers.
- **chrF** (Popović, 2015) for round-trip translation (English → Hindi/Telugu → English) and number preservation.
- **Latency** in milliseconds for analysis, summary and Q&A.

5.2 RESULT SUMMARY TABLES

Table 5.1: Information extraction and Q&A (rules only vs with Hugging Face models)

Metric	Dev: rules	Dev: models	Held-out: rules	Held-out: models
Decisions P / R / F1	1.0 / 1.0 / 1.0	1.0 / 1.0 / 1.0	0.67 / 1.0 / 0.80	0.67 / 1.0 / 0.80
Decision revisions detected	1/1	1/1	0/0	0/0
Proposals P / R / F1	1.0 / 1.0 / 1.0	1.0 / 1.0 / 1.0	1.0 / 0.33 / 0.50	1.0 / 0.33 / 0.50
Action items P / R / F1	1.0 / 1.0 / 1.0	1.0 / 1.0 / 1.0	1.0 / 1.0 / 1.0	1.0 / 1.0 / 1.0
Task owner / deadline accuracy	1.0 / 1.0	1.0 / 1.0	1.0 / 1.0	1.0 / 1.0
Unresolved questions F1	1.0	1.0	1.0	1.0
Q&A answerable correct	12/12	12/12	4/4	4/4
Q&A unanswerable → abstained	5/5	5/5	2/2	2/2

Table 5.2: Summary quality (ROUGE F1) and faithfulness

Metric	Dev: TextRank	Dev: abstractive	Held-out: TextRank	Held-out: abstractive
Overview ROUGE-1 / 2 / L	0.449 / 0.327 / 0.425	0.555 / 0.336 / 0.405	0.357 / 0.116 / 0.242	0.550 / 0.367 / 0.430
Full catch-up ROUGE-1 / L	0.382 / 0.314	0.348 / 0.275	0.458 / 0.249	0.408 / 0.247
Abstractive sentences evidence-supported	–	12/12	–	4/4

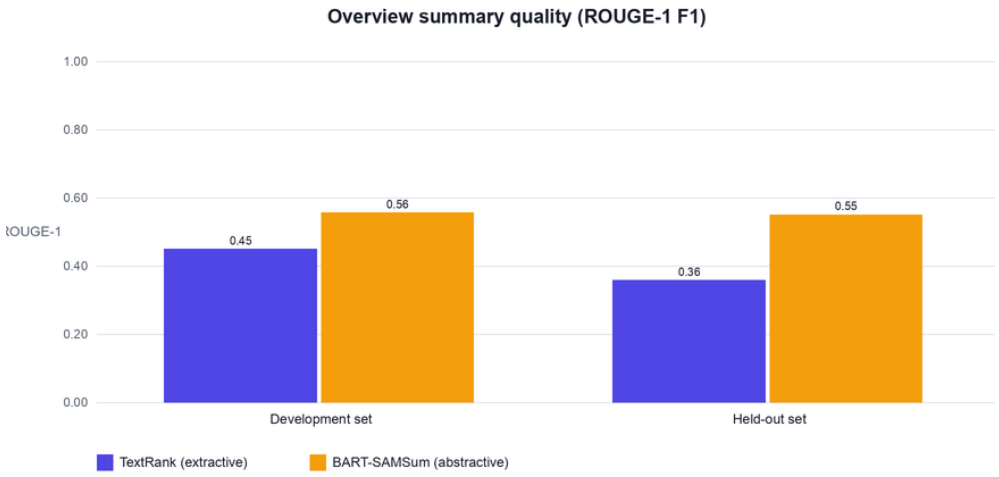


Figure 5.1: Overview summary quality: extractive TextRank vs abstractive BART-SAMSum

Table 5.3: Summarizer selection benchmark (development set)

Model	Overview ROUGE-1	ROUGE-L	Sentences supported	Seconds / meeting
philschmid/bart-large-cnn-samsum (chosen)	0.554	0.410	16/16	5.31
knkarthick/MEETING_SUMMARY	0.514	0.375	10/10	4.63

Table 5.4: Dialogue-act classification on 32 labelled sentences

Act	Cue-phrase rules (recall)	Zero-shot NLI (recall)
decision	8/8	0/8
proposal	4/6	5/6
task	6/6	2/6
question	5/5	5/5
other	7/7	7/7
Accuracy	**0.938**	**0.594**

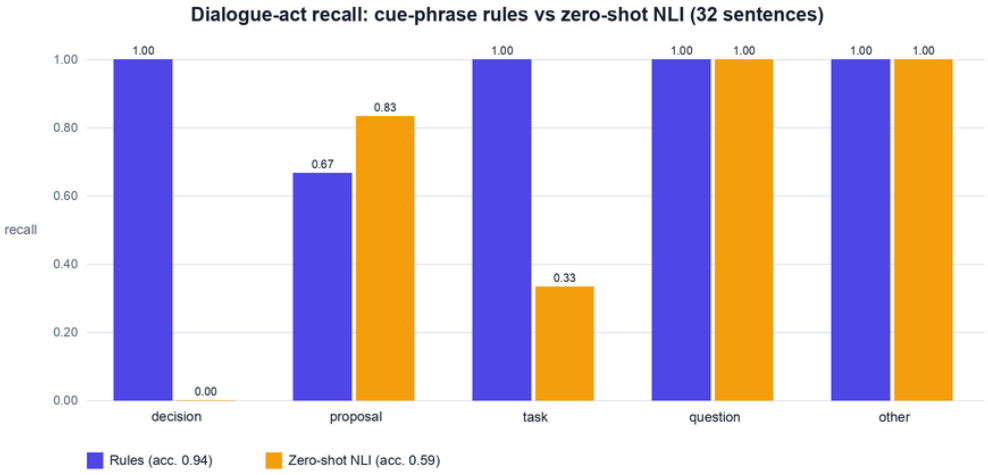


Figure 5.2: Dialogue-act recall of cue-phrase rules versus zero-shot NLI

Table 5.5: Explanation model benchmark (9 items, human-rated)

Model	Parameters	Correct	Partly	Wrong	Typical latency
google/flan-t5-large	780 M	0	2	7	1–2 s
Qwen/Qwen2.5-0.5B-Instruct	0.5 B	2	4	3	2–6 s
Qwen/Qwen2.5-1.5B-Instruct (chosen)	1.5 B	**8**	1	0	5–15 s

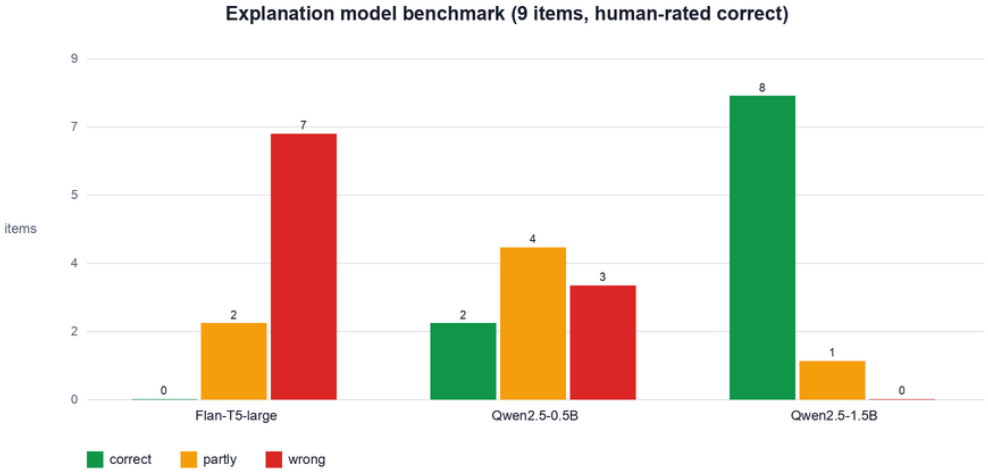


Figure 5.3: Explanation quality of the three candidate models

Table 5.6: Speech and translation

Component	Metric	Result
ASR (Whisper base)	WER on 29.3 s synthetic 3-voice meeting	0.0 (1.6 s to transcribe)
Diarization (WavLM + online clustering)	Speaker accuracy; speakers found	1.0; 3 of 3 (1.2 s)
Translation en→hi→en (NLLB-200)	Round-trip chrF; numbers preserved	0.835; 6/6 (1.22 s / sentence)
Translation en→te→en (NLLB-200)	Round-trip chrF; numbers preserved	0.827; 6/6 (0.88 s / sentence)

Table 5.7: Latency in milliseconds (held-out meeting, CPU)

Operation	Rules only	With Hugging Face models
Analyse meeting (memory)	27.0	1,071.0
Catch-up summary	0.9	5,798.4
Question answering	4.2	54.1

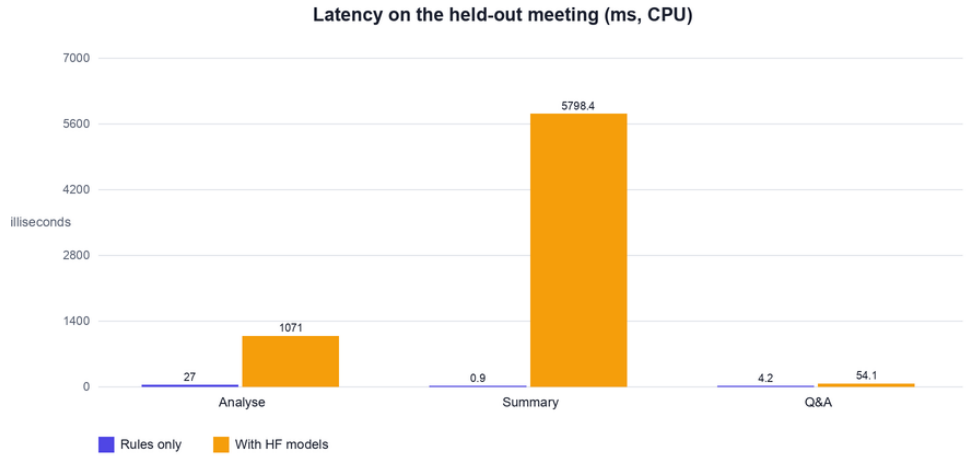


Figure 5.4: Latency of the rule-based and model-based configurations

5.2.1 LIVE MULTI-PARTICIPANT TRIAL

A live test was run with two laptops in one Google Meet call, with the backend on one laptop and the second laptop connecting over Wi-Fi as a room member. The trial confirmed the end-to-end flow — both laptops joined the same room, exchanged heartbeats every 10 s, one laptop captured captions for the room while the other stood by and read the shared summary and catch-up — and exposed several integration problems that the simulated tests had not shown (Table 5.8). All of them were fixed and covered by new harness checks.

Table 5.8: Issues found in the live trial and their fixes

Observed problem	Root cause	Fix
No Share prompt; extension never detected the call	Meet served the meeting at /meet/<code>; only /<code> was recognised	Meeting code matched anywhere in the path; participant tiles as a second in-call signal
Captured text "Summarise captions close"	Meet places buttons and icon glyphs inside the caption region	Caption reader removes button and icon text before emitting speech
Two laptops captured separately	Room lookup happened only once at join time	Look for a room on every 10 s heartbeat
Member left and re-joined repeatedly	Meet hides its toolbar when the mouse is idle	Leave requires 6 s without call controls
Identity ignored after a backend restart	A failed server check was cached for 60 s	Saved identity always sent; failures not cached
Status updates flooded the server	"No captions" status re-sent every 0.7 s	Status reported only on change

5.3 ANALYSIS OF RESULTS

Extraction. Rule-based extraction of action items, owners and deadlines is precise and generalises to the held-out meeting (F1 1.0), because task language is formulaic ("please … by Friday", "I will …"). Decision precision drops to 0.67 on held-out data: the rules confirmed one statement that annotators considered a proposal. Proposal recall (0.33) is the weakest point — suggestions are phrased in many indirect ways ("what if we…", "maybe the queue could…") that cue phrases miss.

Models versus rules. Adding models did not change extraction scores, which is expected because the models only confirm rule-based items. Zero-shot NLI was clearly worse for dialogue acts (accuracy 0.59 vs 0.94): it never recognised a decision and classified most decisions as suggestions, and in an earlier experiment it added false decisions, lowering held-out decision precision from 0.67 to 0.50. This justified keeping NLI optional.

Summaries. The abstractive overview improves ROUGE-1 by about 0.10 on the development set and 0.19 on the held-out set over TextRank, and all generated sentences were supported by transcript evidence. For the *full* catch-up, ROUGE is slightly lower with the abstractive overview, because the catch-up is dominated by quoted key moments and structured lists and the paraphrased overview shares fewer exact words with the reference. ROUGE rewards word overlap rather than correctness, so evidence alignment is reported alongside it.

Q&A. All answerable questions were answered correctly and all unanswerable questions were refused, including "why" questions without a stated reason. The live demo also revealed that a question about a revised plan first returned the superseded value; re-running the reader on the revising decision fixed this without changing the benchmark scores.

Explanations. Model size mattered most: Flan-T5-large invented domain meanings (for example describing sharding as file compression), Qwen2.5-0.5B was often vague, and Qwen2.5-1.5B was correct on 8 of 9 items. Its latency (5–15 s) is acceptable for an on-demand feature.

Speech and translation. WER 0.0 and perfect speaker accuracy on clean synthetic TTS audio show that the pipeline works, but real meetings with overlap and noise will be harder. Round-trip chrF above 0.8 and preserved numbers indicate that meaning survives translation, but human review is still needed for fluency.

Latency. With models, analysis takes about one second and a catch-up summary about six seconds on a laptop CPU; the rule-only configuration answers in milliseconds, which makes it a useful fallback on weak machines.

5.3.1 COMPARISON WITH EXISTING METHODS

Table 5.9 compares MeetBrief qualitatively with earlier research systems and with typical commercial cloud meeting assistants. Quantitative comparison with published ROUGE scores is not meaningful here, because those are reported on AMI, ICSI, SAMSum or QMSum test sets while this project's numbers come from small synthetic meetings.

Table 5.9: Comparison with existing approaches

Capability	**Classical research systems (e.g. CALO, AMI baselines)**	**Typical commercial cloud assistants**	**MeetBrief AI**
Where processing runs	Server / offline	Vendor cloud	User's own laptop (local models)
Cost	Research prototypes	Subscription	Free, open-source models
Structured decisions, tasks, owners	Yes (supervised classifiers)	Usually action items	Yes, with revision history and evidence
Hallucination control	Extractive only	Varies, rarely shown	Evidence alignment + abstention
Late joiner catch-up	No	Usually only for the host's account	Shared rooms, per-member "what you missed"
Indian languages / code-switching	No	Limited	Telugu, Kannada, Hindi, Tamil, Hinglish, Telugu–English
Explain a statement simply	No	Generic chat	Explain Again: 5 styles × 4 levels, grounded
Works without platform captions	Needs recordings	Platform-dependent	Tab audio, desktop recorder, transcript import

Chapter 6: Inferences

6.1 KEY FINDINGS

- **Hybrid beats either extreme.** Transparent rules give precise, explainable decisions and tasks; models add fluency (summaries, explanations, translation) and confidence; evidence alignment keeps model output honest.
- **Zero-shot NLI is not a drop-in classifier for meeting acts.** It cannot reliably distinguish deciding from suggesting, so it should only confirm rule-based items.
- **Abstention is essential and achievable.** Combining a relevance gate with SQuAD 2.0's no-answer score refused every unanswerable question in the evaluation without losing answerable ones.
- **Model size matters for explanations.** A 1.5 B-parameter instruction-tuned model was the smallest that gave reliably correct explanations; smaller models hallucinated domain meanings.
- **Late-joiner support is a coordination problem, not an NLP problem.** The information must be captured by someone; shared rooms keyed by the meeting link, one elected capturer and per-member join times solve it cleanly.
- **Simulated tests are necessary but not sufficient.** The live trial found six integration issues in DOM detection and coordination that no unit test had covered.
- **A local assistant is practical.** All features run on a student laptop with response times of a few seconds.

6.2 FULFILLMENT OF OBJECTIVES

Table 6.1: Fulfillment of the objectives

Objective	Status	Evidence
O1 Capture	Achieved (Meet); experimental (Zoom/Teams web)	Live Meet capture; tab audio; desktop recorder; transcript import; 17-check harness
O2 Understanding	Achieved	Action-item F1 1.0, decision F1 0.80 (held-out), revisions, questions F1 1.0
O3 Faithful summaries	Achieved	ROUGE-1 0.55 abstractive; 16/16 sentences evidence-supported
O4 Grounded Q&A	Achieved	16/16 answerable correct, 7/7 unanswerable abstained
O5 Explain and translate	Achieved	Qwen2.5-1.5B 8/9 correct; hi/te chrF 0.84/0.83
O6 Late joiners	Achieved	Shared rooms verified in live two-laptop trial; 23 room tests; 12-check harness
O7 Privacy and security	Achieved	Local models, private mode, fail-closed access control, DAST scan with 0 open findings
O8 Evaluation	Achieved on small data	Full benchmark; larger corpora (AMI/QMSum) left for future work

The objectives were met on the project's own data and in the live trial. Two qualifications remain: the Zoom and Teams web adapters were implemented but not tested in live calls, and the accuracy figures come from small synthetic meetings, so performance on long, noisy, real meetings still has to be measured.

Chapter 7: Impression Report

7.1 PERSONAL LEARNING EXPERIENCE

This project was the first time I built an NLP system end to end, from reading a meeting page in the browser to evaluating each component with proper metrics. The most important lesson was that a model's output is only useful when it can be verified: adding evidence ids to every sentence, and letting the system say "I don't know", made the assistant trustworthy in a way that a bigger model alone would not. A second lesson came from the live test with a friend's laptop: features that passed every automated test failed on the real Google Meet page because of a URL variant, hidden toolbars and buttons inside the caption area. Debugging these from server logs taught me how to reason about a distributed system from its observable behaviour. Personalise this section with your own experience.

7.2 SKILL ENHANCEMENT

- **NLP and machine learning:** Transformers for summarization, QA, NER, NLI, translation and generation; sentence embeddings and hybrid retrieval; classical TextRank, TextTiling and TF-IDF; designing fair rules-vs-models evaluations and choosing models from benchmarks rather than assumptions.
- **Speech processing:** Whisper ASR, speaker embeddings and online clustering for diarization.
- **Software engineering:** Chrome MV3 extensions, FastAPI, SQLite, concurrency (serialising state updates and database access), and writing 88 automated tests and browser harnesses.
- **Security:** authentication and authorization design, broken object-level authorization tests, CSP, and dynamic application security testing with HawkScan.
- **Research and communication:** reading the literature, documenting limitations honestly, and writing this report.

7.3 INDUSTRY / ACADEMIC RELEVANCE

Meeting intelligence is an active industry area, and privacy regulations make on-device processing increasingly important for companies, hospitals and educational institutions. The hybrid, evidence-grounded design is directly relevant to trustworthy AI, where explainability and abstention are required. Academically, the project connects core NLP course topics — tokenisation, language identification, classification, information extraction, summarization, question answering, machine translation and evaluation — in one working system, and it provides an evaluation framework that can be extended to public corpora such as AMI and QMSum.

Chapter 8: Conclusion & Future Work

MeetBrief AI shows that a useful meeting assistant can be built from open models running on a student laptop, without sending meeting content to any external AI service. The system captures Google Meet captions, tab audio, desktop audio or imported transcripts; builds a structured meeting memory; produces evidence-aligned summaries; answers questions with quoted evidence and abstains when information is missing; explains and translates content into each viewer's language; and gives late joiners a personalised catch-up through shared rooms.

The main take-away is methodological. Precise structured information (who must do what by when) is best extracted with transparent rules, while models are most valuable for fluent text and should be constrained by evidence. On the evaluation data, action items reached F1 1.0, decisions 0.80 on held-out data, Q&A was 16/16 correct with 7/7 correct abstentions, and the abstractive overview raised ROUGE-1 from 0.36 to 0.55 with every sentence supported. Benchmarks guided the model choices: rules over zero-shot NLI for dialogue acts, BART-SAMSum over a meeting-specific summarizer, and Qwen2.5-1.5B over Flan-T5 for explanations.

The work also has clear limitations. Kannada speech is the weakest language, Telugu and Kannada appear in English only, the capturing user's own voice needs captions, and a full live five-language test is still pending. The datasets are small and synthetic, so the reported numbers are not evidence of performance on long, noisy, real meetings. Proposal detection has low recall. The Zoom and Teams web adapters are untested in live calls, and caption reading depends on each platform's page structure, which changes over time. A late joiner benefits only if at least one participant with the extension was present from the start or an official transcript exists. Small local models are slower and less fluent than large hosted models.

FUTURE WORK

- Evaluate on public corpora (AMI, ICSI, QMSum) and on real recorded Indian code-switched meetings with human judgements of faithfulness, explanation clarity and translation fluency.
- Improve proposal recall with a small supervised dialogue-act classifier fine-tuned on AMI dialogue-act annotations, keeping rules for precision.

- Add a server-side meeting bot (Google Meet Media API, Zoom Meeting SDK or Teams bots) so that capture does not depend on any participant's browser.
- Verify and harden the Zoom and Teams web adapters, and provide published OAuth apps so transcript import does not require pasted tokens.
- Use overlapped-speech detection and end-to-end diarization for real noisy audio, and evaluate DER.
- Offer organisation deployment: HTTPS server, Chrome Enterprise force-install, and verified Google sign-in for all members.
- Quantise models (int8 / 4-bit) to reduce latency and memory on low-end laptops.

REFERENCES

Carletta, J., Ashby, S., Bourban, S., Flynn, M., Guillemot, M., Hain, T., Kadlec, J., Karaiskos, V., Kraaij, W., Kronenthal, M., Lathoud, G., Lincoln, M., Lisowska, A., McCowan, I., Post, W., Reidsma, D., & Wellner, P. (2005). The AMI meeting corpus: A pre-announcement. In *Machine Learning for Multimodal Interaction (MLMI 2005)*, LNCS 3869 (pp. 28–39). Springer.

Chen, S., Wang, C., Chen, Z., Wu, Y., Liu, S., Chen, Z., Li, J., Kanda, N., Yoshioka, T., Xiao, X., Wu, J., Zhou, L., Ren, S., Qian, Y., Qian, Y., Wu, J., Zeng, M., Yu, X., & Wei, F. (2022). WavLM: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing, 16*(6), 1505–1518.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019* (pp. 4171–4186).

Gliwa, B., Mochol, I., Biesek, M., & Wawer, A. (2019). SAMSum corpus: A human-annotated dialogue dataset for abstractive summarization. In *Proceedings of the 2nd Workshop on New Frontiers in Summarization* (pp. 70–79).

He, P., Gao, J., & Chen, W. (2023). DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *International Conference on Learning Representations (ICLR 2023)*.

Hearst, M. A. (1997). TextTiling: Segmenting text into multi-paragraph subtopic passages. *Computational Linguistics, 23*(1), 33–64.

Hsueh, P.-Y., & Moore, J. D. (2007). Automatic decision detection in meeting speech. In *Machine Learning for Multimodal Interaction (MLMI 2007)*, LNCS 4892 (pp. 168–179). Springer.

Janin, A., Baron, D., Edwards, J., Ellis, D., Gelbart, D., Morgan, N., Peskin, B., Pfau, T., Shriberg, E., Stolcke, A., & Wooters, C. (2003). The ICSI meeting corpus. In *Proceedings of IEEE ICASSP 2003* (pp. I-364–I-367).

Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2020a). BART: Denoising sequence-to-sequence

pre-training for natural language generation, translation, and comprehension. In *Proceedings of ACL 2020* (pp. 7871–7880).

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020b). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33* (pp. 9459–9474).

Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out* (pp. 74–81). Association for Computational Linguistics.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.

Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On faithfulness and factuality in abstractive summarization. In *Proceedings of ACL 2020* (pp. 1906–1919).

Mihalcea, R., & Tarau, P. (2004). TextRank: Bringing order into texts. In *Proceedings of EMNLP 2004* (pp. 404–411).

Murray, G., Renals, S., & Carletta, J. (2005). Extractive summarization of meeting recordings. In *Proceedings of Interspeech 2005* (pp. 593–596).

NLLB Team, Costa-jussà, M. R., Cross, J., Çelebi, O., Elbayad, M., Heafield, K., Heffernan, K., et al. (2022). No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*.

Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The PageRank citation ranking: Bringing order to the web* (Technical Report 1999-66). Stanford InfoLab.

Park, T. J., Kanda, N., Dimitriadis, D., Han, K. J., Watanabe, S., & Narayanan, S. (2022). A review of speaker diarization: Recent advances with deep learning. *Computer Speech & Language, 72*, 101317.

Popović, M. (2015). chrF: Character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation* (pp.

392–395).

Purver, M., Dowding, J., Niekrasz, J., Ehlen, P., Noorbaloochi, S., & Peters, S. (2007). Detecting and summarizing action items in multi-party dialogue. In *Proceedings of the 8th SIGdial Workshop on Discourse and Dialogue* (pp. 18–25).

Qwen Team. (2024). Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, PMLR 202 (pp. 28492–28518).

Rajpurkar, P., Jia, R., & Liang, P. (2018). Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of ACL 2018* (pp. 784–789).

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP 2019* (pp. 3982–3992).

Reimers, N., & Gurevych, I. (2020). Making monolingual sentence embeddings multilingual using knowledge distillation. In *Proceedings of EMNLP 2020* (pp. 4512–4525).

Snyder, D., Garcia-Romero, D., Sell, G., Povey, D., & Khudanpur, S. (2018). X-vectors: Robust DNN embeddings for speaker recognition. In *Proceedings of IEEE ICASSP 2018* (pp. 5329–5333).

Stolcke, A., Ries, K., Coccaro, N., Shriberg, E., Bates, R., Jurafsky, D., Taylor, P., Martin, R., Van Ess-Dykema, C., & Meteer, M. (2000). Dialogue act modeling for automatic tagging and recognition of conversational speech. *Computational Linguistics, 26*(3), 339–373.

Tjong Kim Sang, E. F., & De Meulder, F. (2003). Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In *Proceedings of CoNLL-2003* (pp. 142–147).

Tur, G., Stolcke, A., Voss, L., Peters, S., Hakkani-Tür, D., Dowding, J., Favre, B., Fernández, R., Frampton, M., Frandsen, M., Frederickson, C., Graciarena, M., Kintzing, D., Leveque, K., Mason, S., Niekrasz, J., Purver, M., Riedhammer, K.,

Shriberg, E., Tien, J., Vergyri, D., & Yang, F. (2010). The CALO meeting assistant system. *IEEE Transactions on Audio, Speech, and Language Processing, 18*(6), 1601–1611.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30* (pp. 5998–6008).

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., & Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP 2020: System Demonstrations* (pp. 38–45).

Yin, W., Hay, J., & Roth, D. (2019). Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach. In *Proceedings of EMNLP-IJCNLP 2019* (pp. 3914–3923).

Zhong, M., Yin, D., Yu, T., Zaidi, A., Mutuma, M., Jha, R., Awadallah, A. H., Celikyilmaz, A., Liu, Y., Qiu, X., & Radev, D. (2021). QMSum: A new benchmark for query-based multi-domain meeting summarization. In *Proceedings of NAACL-HLT 2021* (pp. 5905–5921).

Zhu, C., Xu, R., Zeng, M., & Huang, X. (2020). A hierarchical network for abstractive meeting summarization with cross-domain pretraining. In *Findings of EMNLP 2020* (pp. 194–203).

Appendix

APPENDIX 1: SELECTED REST API ENDPOINTS

Table A.1: Selected backend endpoints

Method and path	Purpose
POST /api/meetings; POST /api/meetings/{id}/segments	Create a meeting; add caption segments
POST /api/meetings/{id}/audio	Transcribe a 15 s audio chunk (Whisper + diarization)
POST /api/upload; POST /api/upload-recording	Import a transcript file; import a recording (background ASR)
POST /api/meetings/{id}/summary	Catch-up / quick / detailed / chronological / topic / personalised summary
POST /api/meetings/{id}/ask	Grounded question answering with evidence and abstention
POST /api/meetings/{id}/explain; POST /api/translate	Explain Again; translation
GET /api/meetings/{id}/decisions | tasks | timeline | speakers	Structured meeting memory
GET /api/meetings/{id}/export?format=md|pdf|txt|json	Export minutes
POST /api/meetings/{id}/finalize	Final summary when the meeting ends
POST /api/rooms/share; GET /api/rooms/lookup	Share a meeting as a room; find the room of a meeting link
POST /api/rooms/{id}/join | heartbeat | segments | leave	Join, capturer election, capturer-only transcript, leave
GET /api/rooms/{id}/catchup	Personalised "what you missed" catch-up
POST /api/import/{google_meet|zoom|teams}	Import the platform's official transcript
DELETE /api/meetings/{id}; DELETE /api/data	Privacy: delete a meeting or all data

APPENDIX 2: PROJECT STRUCTURE AND HOW TO RUN

```
nlp/
  chrome-extension/  MV3 extension: background.js, content/, sidepanel/,
                     audio/, lib/identity.js
  backend/           FastAPI app: api/, auth.py, capture/, nlp/, rag/,
                     memory/, speech/, hf/, database/, connectors/,
                     export/, evaluation/, tests/
  tools/             desktop_recorder.py
  datasets/          sample transcripts, annotations, synthetic audio
  research/          measured evaluation results and benchmarks
  docs/              architecture, user guide, rooms, evaluation, security

# start the backend (first run creates .venv and installs dependencies)
./run.sh
# shared rooms on a LAN with test identities
MEETBRIEF_HOST=0.0.0.0 MEETBRIEF_ROOMS_DEV_AUTH=1 ./run.sh
# tests and evaluation
.venv/bin/python -m pytest backend/tests -q
.venv/bin/python -m backend.evaluation.run_eval
# Chrome: chrome://extensions > Developer mode > Load unpacked > chrome-extension/
```

APPENDIX 3: EXAMPLE CATCH-UP OUTPUT (UNEDITED)

The catch-up below was generated by MeetBrief for the sprint-planning meeting; bracketed ids such as [S3] link each line to the transcript segment that supports it.

Main problem / goal

- Karthik: The main issue is that we load all the payment provider scripts up front. [S3]

Overview *(abstractive — bart-large-cnn-samsum, verify flagged items)*

- The payment page needs to be lazy loaded only when the user selects a payment method and the address validation should be moved to the backend. [S3] The database migration needs a rollback plan and Sana will write it by next Monday. [S19] The mobile app release for 2.4 will be postponed to next Tuesday. [S13]

Decisions

- Go with lazy loading the payment scripts first, because it is the smallest change with the biggest impact (by Meera) [S5]
- Postpone the 2.4 release to next Tuesday instead, so David has time to fix the Android crash (by Meera) [S13]

Action items

- Implement the lazy loading — owner: Karthik; deadline: Wednesday [S6]
- Write the rollback plan — owner: Sana; deadline: next Monday [S19]